Contents

System Design — Motor Insurance Claims Settlement System **1**
1. Project overview and objectives . . . 1
2. System architecture . . . 1
3. Python modules and responsibilities . . . 3
4. Input data flow . . . 3
5. Claim assessment workflow . . . 4
6. Depreciation and settlement processing . . . 4
7. Theft settlement workflow . . . 4
8. NCB and policy renewal workflow . . . 5
9. Report generation . . . 5
10. Error handling . . . 5
11. Technologies and libraries . . . 5
12. Business-rule traceability matrix . . . 6
    Verification status . . . 7
13. Detailed claim calculation flowchart . . . 8
14. Front-end addition . . . 8

System Design — Motor Insurance Claims Settlement System

1. Project overview and objectives

The PS-011 project is a Python command-driven motor-insurance claims settlement application for Team T051. It loads policy, claim, and survey-line data from CSV files, applies the implemented claim and renewal rules, and writes claim, policy-history, and validation/error reports as CSV files.

The system's objectives, as evidenced by the implementation, are to:

- validate and retain policy, claim, and survey data;
- assess repair, total-loss, and theft claims using the configured rules;
- calculate policy-year NCB at renewal; and
- produce repeatable, tabular computation and history reports.

2. System architecture

The application separates command orchestration, data models and state, validation, claim calculations, and report generation. `src/main.py` is the entry point. `CommandProcessor` executes the command file in order and passes shared `ApplicationState` through loaders and calculation functions.

Editable Graphviz source: `system_architecture.dot`.

The implemented theft command workflow is in `command_processor.py`; the separate `theft.py` module also exposes theft eligibility and settlement helpers. The command processor currently performs its command-level theft handling in its own methods.

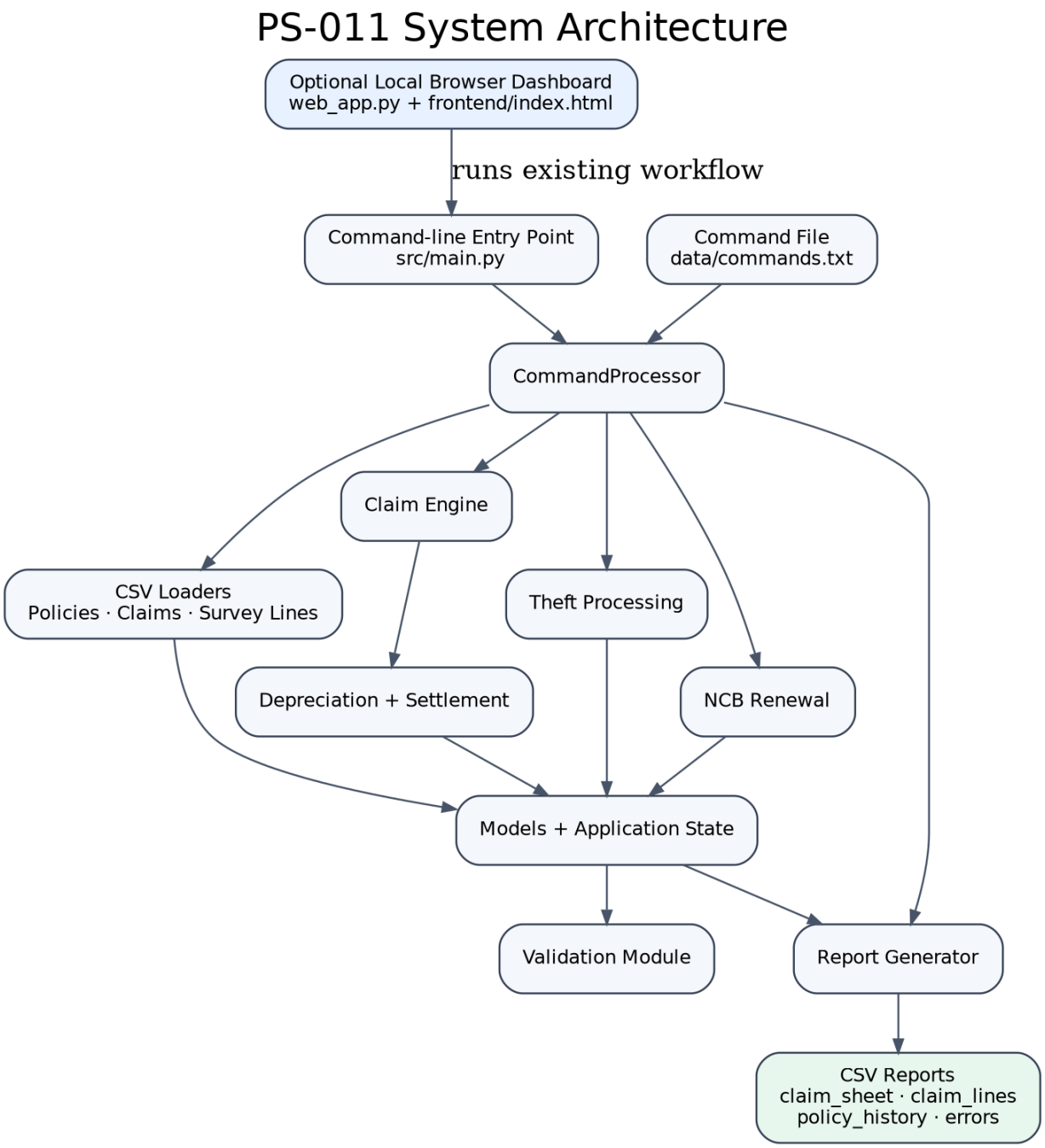


Figure 1: System architecture diagram

3. Python modules and responsibilities

Module	Responsibility
src/__init__.py	Marks src as a Python package.
src/main.py	Resolves the project root, runs command processing, generates reports, prints execution totals, and returns a process status.
src/command_processor.py	Parses and executes commands in file order; invokes loaders, claim assessment, theft actions, and renewals; queues report requests and records command errors.
src/models.py	Defines the Policy, Claim, and SurveyLine dataclasses and their field types.
src/state.py	Holds loaded records, claim results, policy history, errors, and theft recovery dates in ApplicationState.
src/constants.py	Centralizes rates, thresholds, categories, add-ons, decisions, statuses, and other rule constants.
src/rule_ids.py	Defines BR-01 through BR-12 descriptions/order and formats rule identifiers.
src/data_loader.py	Parses policy, claim, and survey CSV rows into model instances and records row-level parsing errors.
src/validators.py	Validates loaded policies, claims, references, and survey lines; rejects invalid lines and exact duplicate survey entries.
src/depreciation.py	Computes completed vehicle age, part depreciation, payable part amounts, and two-decimal monetary rounding.
src/settlement.py	Calculates compulsory/total excess, line coverage, covered repair-line amounts, and repair settlement totals.
src/claim_engine.py	Coordinates non-theft claim checks, ZERO_DEP eligibility, total-loss detection, repair/total-loss settlement, and applied rule IDs.
src/theft.py	Provides theft FIR timing, waiting-period and recovery checks, RTI eligibility, and theft settlement helper functions.
src/ncb.py	Computes the next NCB slab, counts policy-period claims and ZERO_DEP usage, and records renewal NCB history.
src/report_generator.py	Builds and writes claim-line, claim-sheet, policy-history, and error CSV reports.

4. Input data flow

1. main() creates a CommandProcessor using the project root. Its data directory is data/ and its report directory is output/.
2. The processor opens data/commands.txt, ignoring blank lines and lines beginning with #, and executes remaining commands in their original order.
3. LOAD_POLICIES, LOAD_CLAIMS, and LOAD_SURVEY use csv.DictReader to parse the respective CSV files. Dates, timestamps, decimals, booleans, and add-ons are converted into Python values.
4. On LOAD_SURVEY, validate_all() checks all loaded records. Valid survey lines are grouped by claim_id in application state.

5. Later assessment, theft, renewal, and print commands operate on that shared state. Data-loading and validation errors are retained for the error report rather than silently discarded.

The field schemas and input validation details are documented in sample_data_README.md.

5. Claim assessment workflow

For a non-theft `ASSESS` command, the processor checks that the claim exists and dispatches to `assess_claim()`. In sequence, the claim engine:

1. Confirms the claim's policy exists and the assessment date is not before the incident.
2. Calculates vehicle age from the purchase date to the incident date.
3. Rejects an intimation more than seven days after the incident; otherwise determines whether a delay deduction applies.
4. Validates the driving-licence status and expiry on the incident date.
5. Confirms the policy covers the incident date.
6. Determines whether ZERO_DEP is available, based on the add-on, vehicle age, and prior eligible claims in the policy year.
7. Sums all surveyed part and labour costs before depreciation to test the total-loss threshold.
8. Sends a total loss to the IDV/RTI settlement path; otherwise calculates covered repair lines and the repair settlement.
9. Stores the decision, calculations, line details, and applicable rule IDs in application state.

Rejected claims have zero payable line amounts. In a repair settlement, uncovered lines are rejected individually while covered lines continue through calculation.

6. Depreciation and settlement processing

`depreciation.py` uses completed calendar months and category-specific rates. Paint is calculated using the metal rate on half its cost. Covered labour is not depreciated; each line's calculated part depreciation and payable part are rounded to two decimal places.

`settlement.py` calculates covered repair-line amounts and combines them with payable labour. Repair GST is added for individual policyholders and omitted for corporate holders. Excess, any applicable delay deduction, and the learner cap follow in that order. The result is classified as `BELOW_THRESHOLD` when it is less than Rs 500.00. The exact business-rule formulas and boundaries are in business_rules.md.

For total loss, the claim engine compares pre-depreciation surveyed repair cost with 75% of IDV. Settlement is based on IDV or eligible invoice value, then applies excess, any customer-retained salvage deduction, delay deduction, and learner cap. It does not derive the total-loss payout from depreciated repair lines.

7. Theft settlement workflow

The command processor keeps theft apart from ordinary `ASSESS` processing:

1. A theft FIR later than 24 hours after the incident creates a rejected claim result.
2. `THEFT_RECOVERED` records the recovery date. Recovery before FIR date plus 90 days converts the claim to a repair assessment; recovery on or after that date does not.
3. `SETTLE_THEFT` before the eligibility date logs a premature command and leaves settlement pending. On or after the date, it validates the general claim rules, then calculates IDV or eligible RTI invoice value less excess. The 10% delayed-intimation deduction and learner-licence 50% cap are applied after excess when applicable.

The theft settlement route does not use repair-part depreciation, repair GST, or the repair minimum-settlement threshold. Details and boundary conditions are in BR-01, BR-02, BR-08 and BR-09 of business_rules.md.

8. NCB and policy renewal workflow

`RENEW` invokes `process_ncb_renewal()` using the expiring policy dates. Assessed claims are counted by incident date within that inclusive policy period; rejected and unassessed claims are excluded, and `BELOW_THRESHOLD` claims are included. NCB advances one slab with no counted claims, retains its current value for one claim protected by NCB_PROTECT, and resets to zero for an unprotected claim or more than one claim. The processor records the old period and NCB values in history, then changes the policy period to start on the renewal date and end 12 calendar months later minus one day.

9. Report generation

After command processing, `report_generator.py` writes UTF-8-with-BOM CSVs under `output/`:

Report	Contents
`claim_lines.csv`	Calculated line details for assessed claims, ordered by claim and line ID.
`claim_sheet.csv`	A computation row for each valid queued `PRINT_CLAIM_SHEET` command.
`policy_history.csv`	A history row for each valid queued `PRINT_POLICY_HISTORY` command.
`errors.csv`	Recorded loader, validation, and command errors.

Print commands are queued while the command stream is processed; the report generator uses that log to decide which claim sheets and policy histories to write.

10. Error handling

- CSV parsing errors are recorded with their source, row, reference ID, and reason. Policy, claim, and survey validation errors are added to shared application state.
- Invalid commands, unknown IDs, and failures during assessment, theft handling, or renewal are recorded as command errors and processing can continue.
- Missing command files and other failures escaping command processing are reported by `main()` as project execution failures and return a non-zero process status.
- Errors are emitted in `output/errors.csv`. The supplied integration tests assert two errors for the sample command/data pack.

11. Technologies and libraries

- **Python** is the implementation language.
- **Python standard library:** `csv`, `dataclasses`, `datetime`, `decimal`, `pathlib`, `inspect`, `calendar`, and `sys` support parsing, data models, dates, monetary arithmetic, filesystem paths, command dispatch, and process status.

- **pytest** is used by the automated test suite.
- **Python standard library** modules `http.server`, `subprocess`, `csv` and `json` provide the optional local browser dashboard in `web_app.py`.
- **HTML, CSS and JavaScript** provide the dashboard interface in `frontend/index.html`.

No third-party runtime dependency is required for the dashboard. The backend calculation engine uses Python modules and CSV files; it does not call an external insurance calculation service or web API.

12. Business-rule traceability matrix

The table maps each implemented business rule to its primary implementation location and the strongest evidence currently present in the repository. “Sample evidence” means the supplied data/command stream exercises the scenario; it is not a substitute for a dedicated assertion of every calculated amount.

Rule	Implementation location	Existing automated test evidence	Sample-data / verification evidence	Coverage note
BR-01 Intimation and delay	`src/claim_engine.py`: `validate_intimation`; `src/settlement.py`: `calculate_delay_deduction`; `src/command_processor.py`: theft FIR and general intimation checks	`test_intimation_after_seven_days_is_rejected`, `test_late_fir_theft_claim_rejected`, `test_theft_settlement_applies_delay_deduction_and_learner_cap`, `test_exactly_48_hours_has_no_delay_deduction`	CLM-003 (exactly 48h), CLM-015 (>7 days), CLM-017 (50h), CLM-018 (late theft FIR)	Theft route applies the 10% deduction; 48h boundary is directly tested.
BR-02 Licence and learner cap	`src/claim_engine.py`: `validate_driving_licence`; `src/settlement.py`: `calculate_learner_cap`; theft settlement in `src/command_processor.py`	`test_learner_licence_claim`; `test_expired_licence_rejected`; `test_theft_settlement_rejects_invalid_driving_licence`; `test_theft_settlement_applies_delay_deduction_and_learner_cap`	CLM-001 learner licence; CLM-006 expired licence	Learner cap amount and theft licence rejection now have direct test assertions.
BR-03 Part depreciation	`src/depreciation.py`: `get_depreciation_rate`, `calculate_line_depreciation`, `calculate_vehicle_age_months`	`test_exactly_six_months_metal_depreciation`, `test_glass_depreciation_is_zero`, `test_depreciation_rates_cover_all_metal_age_bands`, `test_paint_depreciation_uses_half_cost`	Survey data includes METAL, PLASTIC, RUBBER, NYLON, GLASS and PAINT categories	Metal age bands and paint’s half-cost calculation are directly tested.
BR-04 ZERO_DEP	`src/claim_engine.py`: `is_zero_dep_applicable`, `count_previous_zero_dep_claims`	`test_zero_dep_first_claim`; `test_third_zero_dep_claim_uses_normal_depreciation`	CLM-007, CLM-011 and CLM-016 exercise first/second/third claims	Add a dedicated test for vehicle age over 5 years and exact eligibility limit.
BR-05 Consumables and engine protection	`src/settlement.py`: `get_line_coverage_status`, `calculate_survey_line`	`test_sample_consumables_and_engine_protect_engine_are_not_paid`	CLM-009 flood engine lines without ENGINE_PROTECT; CLM-010 consumables without CONSUMABLES	Both excluded part and labour amounts are directly asserted as zero.
BR-06 Compulsory and voluntary excess	`src/settlement.py`: `calculate_compulsory_excess`, `calculate_total_excess`	`test_compulsory_excess_boundary_petrol_cc`	Sample policies exercise lower/higher engine capacities and voluntary excess	Exactly 1500cc and 1501cc are directly tested.

Rule	Implementation location	Existing automated test evidence	Sample-data / verification evidence	Coverage note
BR-07 Total loss and salvage	`src/claim_engine.py`: `calculate_assessed_repair_cost`, `calculate_total_loss_settlement`	`test_total_loss_at_exact_75_percent`, `test_sample_total_loss_salvage_and_rti_amounts`	CLM-005 is exactly 75% of IDV and customer keeps salvage	Decision, 10% salvage and final payable are directly asserted.
BR-08 RTI	`src/claim_engine.py`: `is_rti_applicable`; theft route in `src/command_processor.py`	`test_sample_total_loss_salvage_and_rti_amounts`	CLM-008 theft with RTI; CLM-005 total-loss scenario without RTI	Invoice settlement basis and amount are asserted. **Known ambiguity:** total-loss and theft paths differ at the exact 12-month anniversary; see `docs/business_rules.md` and `docs/decision_log.md`.
BR-09 Theft settlement and recovery	`src/command_processor.py`: `theft_fir_is_late`, `get_theft_eligibility_date`, `process_theft_recovery`, `settle_theft`	`test_theft_claim_settlement`; `test_premature_theft_command_does_not_prevent_later_settlement`, `test_late_fir_theft_claim_rejected`; theft general-rule regression tests	CLM-004 recovery on day 90, CLM-008 premature then eligible settlement	Day-90 settlement and early-premature handling are covered by the integration suite.
BR-10 NCB and NCB_PROTECT	`src/ncb.py`: `calculate_next_ncb`, `count_policy_year_claims`, `process_ncb_renewal`	`test_no_claim_policy_gets_ncb_increase`, `test_ncb_protect_only_preserves_one_claim`; renewal tests in `test_policy_renewal.py`	POL-005 has NCB_PROTECT with two claims; POL-010 has no claims	One protected claim retains NCB; two claims reset it, both directly tested.
BR-11 Labour and GST	`src/settlement.py`: `calculate_survey_line`, `calculate_repair_settlement`	`test_repair_settlement_applies_gst_and_covered_labour`	Sample data includes individual and corporate policyholders and covered/uncovered labour lines	Individual 18% GST, corporate zero GST and full covered labour are directly tested.
BR-12 Minimum settlement	`src/settlement.py`: `calculate_repair_settlement`	`test_below_threshold_claim`; `test_exact_500_rupees_is_not_below_threshold`, `test_sample_below_threshold_claim_is_recorded_for_ncb`	CLM-003 calculates Rs 499.99 and remains counted for NCB	Both Rs 499.99 and exactly Rs 500.00 are covered.

Verification status

- The current automated suite passes **40 tests** in the checked project environment. Added tests cover theft-claim general checks, metal depreciation age bands, PAINT depreciation, the 1,500 cc excess boundary, NCB_PROTECT limits, GST/labour, exact Rs 500 threshold, RTI and salvage calculations, and uncovered survey lines.
- The supplied sample command stream processes 18 claims, generates claim-line, claim-sheet, policy-history and error reports, and intentionally records a duplicate survey line and an unknown claim ID.
- The table distinguishes tests that assert a result from sample scenarios that merely exercise a path. Not every calculation boundary has an independent assertion yet.
- The exact 12-month RTI anniversary interpretation remains unresolved in the inspected project materials. It is recorded as an open interpretation rather than changed without an authoritative expected result.

13. Detailed claim calculation flowchart

The flowchart below shows the implemented processing order. Theft claims use the separate theft path; repair and total-loss claims share the initial policy, intimation and licence checks.

Editable Graphviz source: `claim_calculation_flowchart.dot`.

14. Front-end addition

A small local browser dashboard is provided by `web_app.py` and `frontend/index.html`. It calls the existing command-line entry point (`python -m src.main`) and displays the CSV reports produced by the same backend. It does not duplicate insurance calculations or replace the command-file interface. This is an optional presentation layer added for demonstration; the original project specification describes a command-file-driven application and lists a GUI as out of scope. It binds to `127.0.0.1` and uses only the Python standard library.

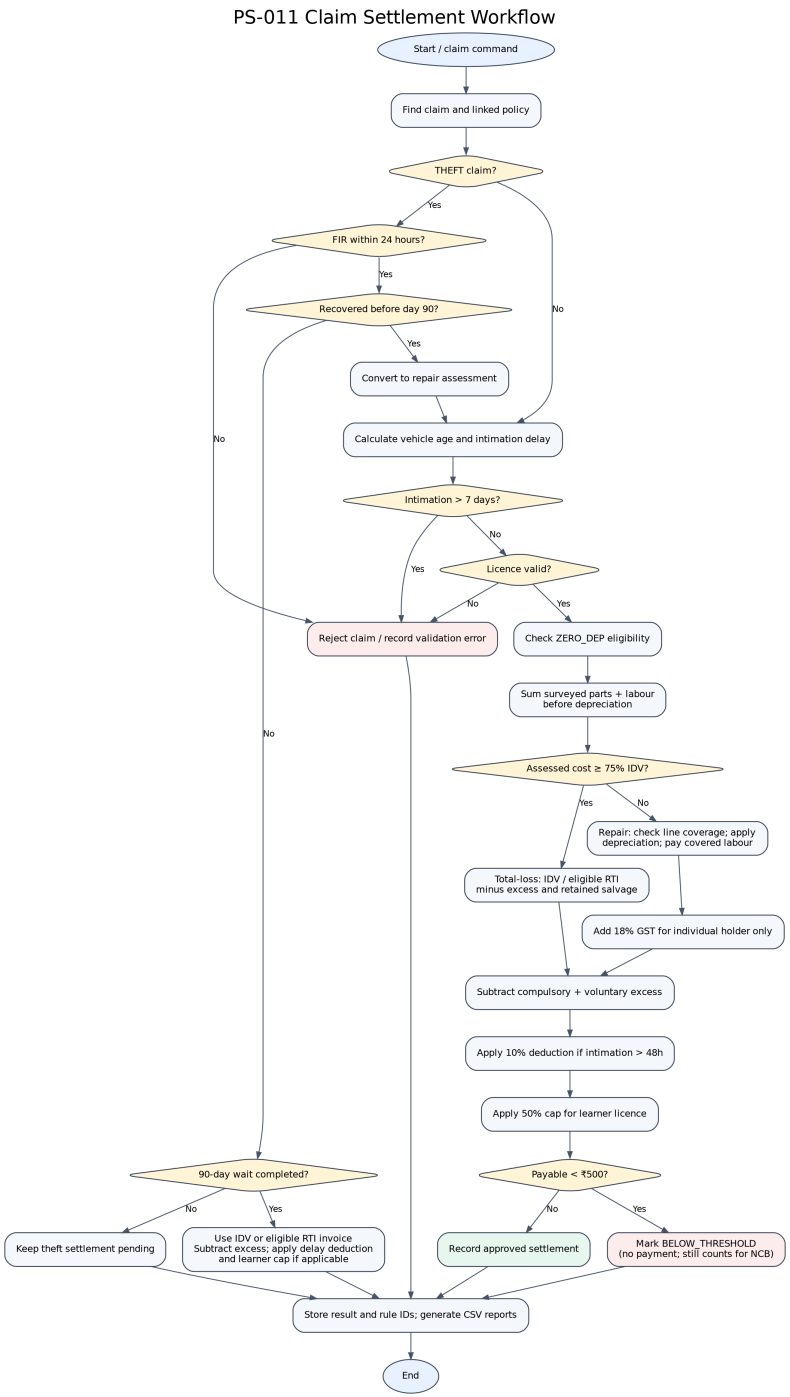


Figure 2: Detailed claim calculation flowchart